Every expression is an lvalue, an xvalue or a prvalue.

Not just two kinds.

```
x                    // lvalue
std::move(x)         // xvalue
x + 1                // prvalue
```

Since C++11 there are three basic categories. The words lvalue and rvalue are groups of them.

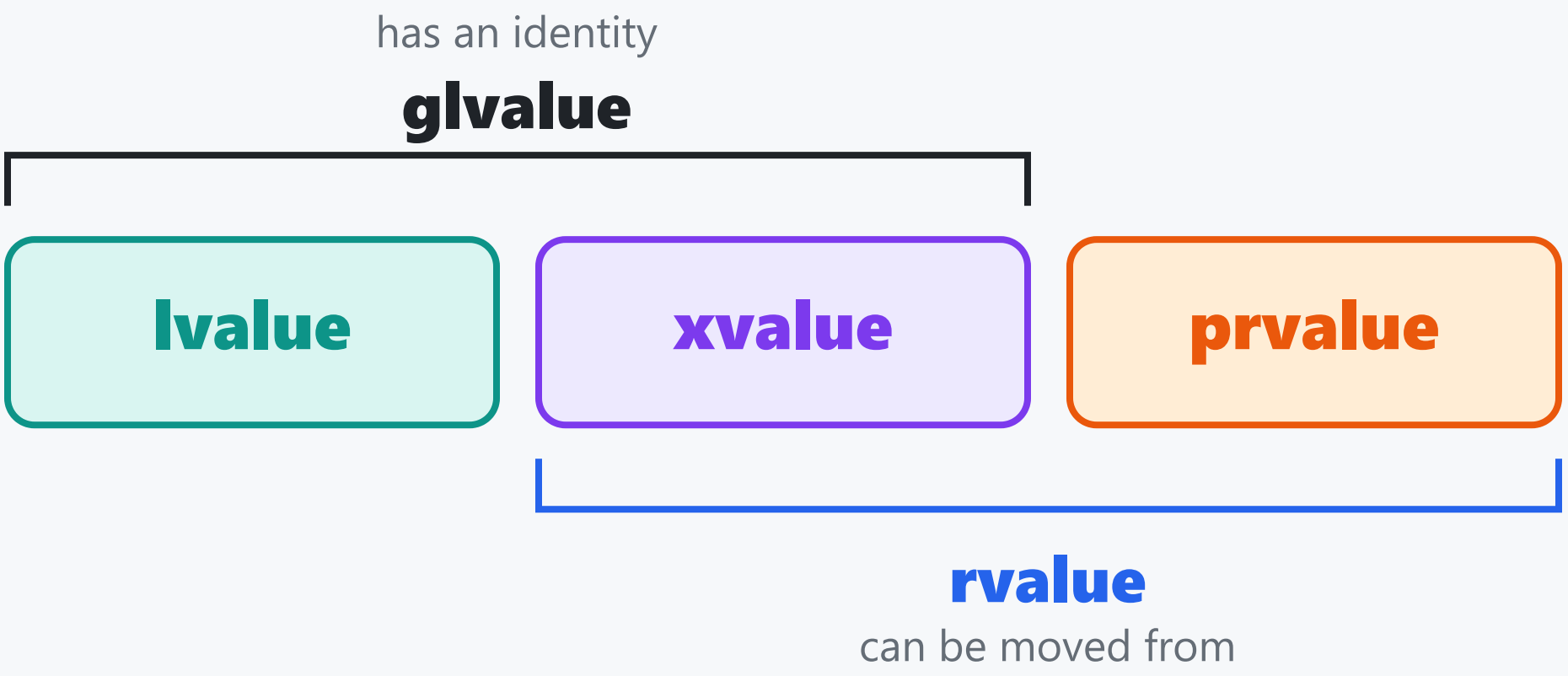
swipe →

The three basic ones

CATEGORY	IT IS	EXAMPLE
lvalue	an object with a name or address	`x` `*p` `arr[0]`
prvalue	a pure temporary value	`5` `x + 1` `make()`
xvalue	an object that is about to be abandoned	`std::move(x)`

`make()` returns a `std::string` by value.

Two groups



The xvalue is the one in both groups: it has an identity and can be moved from.

Ask the compiler

```
decltype((x))            // int&   lvalue
decltype(std::move(x))   // int&&  xvalue
decltype(x + 1)          // int    prvalue
```

RESULT TYPE	CATEGORY
`T&`	lvalue
`T&&`	xvalue
`T`	prvalue

Checked with `static_assert`. The double parentheses matter for a bare name.

Why it matters

REFERENCE	BINDS TO
`T&`	lvalues
`const T&`	everything
`T&&`	xvalues and prvalues

Overload resolution reads the category to pick copy or move.

Since C++17 a prvalue builds its object directly where it will live: no temporary, no move.

One more catch

```
void take(std::string&& s) {
    // s is an lvalue here
}

take(make());
```

The type is an rvalue reference. The **expression** `s` is an lvalue.

A name always makes an lvalue. That is why you write `std::move(s)` again to pass it on.

Which of the three did you not know?